

Bot or Not

Detection of Social Media Bots on X (formerly Twitter)

Benjamin Roderick¹, Derek Ruths, Prof.^{1†}

¹McGill University

[†] Supervisor

benjamin.roderick@mail.mcgill.ca

Abstract

This paper explores the tug-of-war between *Social Media Bots* and platform moderators attempting to detect them. Using the novel approach of exploring the strategies used by both sides of the struggle through an adversarial competition-like format, we are better positioned to make conclusions about the nature of modern *Social Media Bots* in the post-LLM era. For the competition, I created both a *Social Media Bot* and a detector. Over the course of several weeks, I incrementally improved each of them, inspiring myself from the weaknesses I was able to find to eventually obtain the final products presented in this report.

1 Introduction

Since its inception, social media has slowly grown to become an omnipresent feature in our day-to-day lives. Though it brings many advantages, such as keeping up to date with friends, efficient communication of information and building communities, the power it grants to bad actors has shown itself to be worrisome. Indeed, social media was one of the main tools used by foreign actors to interfere in the most recent Canadian election (NSICOP 2024). Unfortunately, bot traffic on social media sites is more active than ever and is more politically inclined than the average human user (Ng and Carley 2025). The growing influence of *Social Media Bots* has created a greater need for tools to detect and ban such users from our social media platforms in an effort to limit the damage they can cause. One problem that all researchers in the field encounter, however, is the fact that, to study *Social Media Bots*, they must first be detected.

In this paper, I will discuss my experience participating in the **Bot or Not** competition organized by Prof. Derek Ruths. The goal of this competition was to create an environment in which multiple students could create *Social Media Bots* and detectors, and have the opportunity to compare their results and techniques amongst themselves. The main advantages of this approach are:

- **Competitor Bots:** Each student must simultaneously create a *Social Media Bot* and a detector. The process of developing a bot forces them to not only be aware of how a *Social Media Bot* functions, but also every new

technique they develop to trick detectors is one that has likely been used by the most advanced bots in real-world situations. Therefore, this feature of the competition directly reduces the impact of one of the biggest limitations to prior works in *Social Media Bots* research, as outlined in Ng and Carley (2025).

- **Transparency:** Since all of the bots in the competition are clearly labeled, it is possible to analyze all of the bots that a detector failed to flag. This feature allows competitors to fine-tune their approaches in situations where there would have otherwise been no information.
- **Experiment Environment:** The competition was organized around a realistic social media environment populated by human user data as well as bot data, along with all the metadata that would be collected by using the real *Twitter* API. This setup encourages competitors to tailor their *Social Media Bots* so as to blend in with the real-world data and thus emulates the difficulty of detecting bots in real datasets, but without the normal drawbacks.

2 Data and Environment

2.1 Data Collection and Processing

All of the data used in the evaluation phase of the competition prior to the content injected by the students' bots was collected from the *Twitter* API. Collected tweets were selected based on the user interacting with a specific hashtag to enforce consistency in subject matter within the data provided to competitors. The data was processed to remove any bots detectable using current means to ensure that the overwhelming majority of the provided "human" users were indeed not *Social Media Bots*.

2.2 Competition Environment

Evaluation of bot and detector performance was conducted in a server environment in which all competitors had uploaded Docker images of their submitted programs. This controlled for differences in computational power between users and ensured equal and reproducible test conditions.

2.3 Training Data and Resources

Each competitor was given an OpenAI API token to use in their *Social Media Bot* to generate content. This allowed

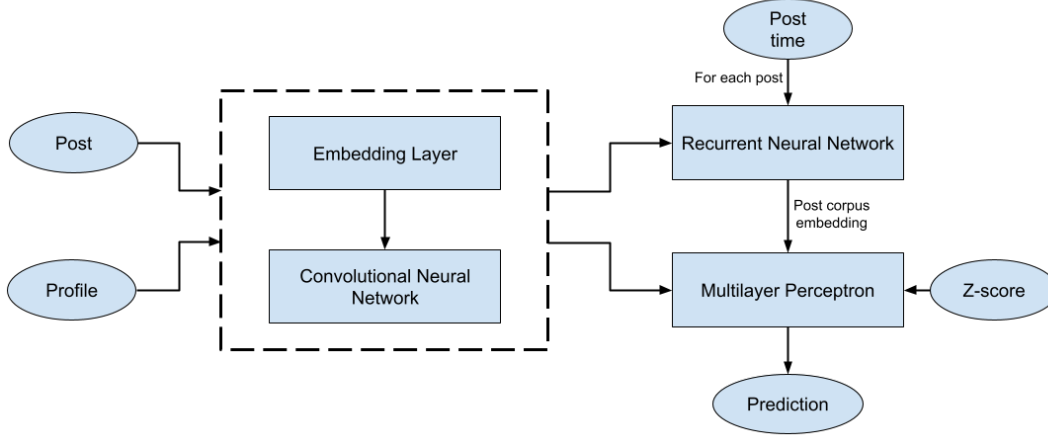


Figure 1: Process diagram of the bot detector

them to create highly sophisticated bots that could adapt to context and even respond to posts from other users. As for the detectors, sample data containing posts, labeled users and metadata was provided at the start of the competition and after each test session. This data was created in .json format and organized according to the following structure:

- **Metadata:** Total number of posts and users, average number of posts and z-score and the hashtags used to generate the dataset.
- **Posts:** A list of all the posts in the dataset, each including the text, unique post id, unique author id, time of post and language.
- **Users:** A list of all the users in the dataset, each item included the unique user id, username, name, description, location and whether the user was a bot or not.

3 Methods

3.1 Research Goals and Ethical Considerations

The primary objective of the competition was to develop new methods to detect *Social Media Bots* while attempting to avoid encountering the limitations that affect related works on the subject. However, the open-ended nature of the research question brings about the necessity to make certain decisions. The strategy employed by competitors, or rather by what measure or combinations of measures they consider a detector to be successful, can have far-reaching impacts if their bot detection strategy were to be adopted by a large social media platform. Indeed, since the only option to deal with a bot once detected is to ban the user, an overzealous detection algorithm would potentially cull nearly all of the bots within the platform’s user base, however, it would also result in a large number of falsely flagged human accounts being banned.

Consequently, the selection of measures is a critical question to be considered during the development of the detection algorithms in the context of the competition. Given the potential adverse consequences of false positives on users,

I decided that the training objective of my detector would be to detect the most bots possible whilst at the same time enforcing the lowest possible false positive rate, even if it came at the expense of detection power. This objective ensures that, while many bots may go undetected, the detector will avoid unnecessary disruption of human users.

3.2 Detector Architecture

Architecture Overview Given the fact that predicting whether or not a user is a bot depends almost entirely on the textual information provided (posts, profile descriptions, usernames), making use of model architectures that are best suited to classifying textual data is critical. In the case of my detector, I elected to use a hybrid model, combining an **Embedding Layer**, a **Convolutional Neural Network**, a **Recurrent Neural Network** and a **Multilayer Perceptron**. As seen in Figure 1, Each component of the model serves a specific purpose in the overall pipeline:

- Using the tokenizer and embedding layer of the open-source DeepSeek-R1-Distill-Qwen-1.5B model (DeepSeek-AI 2025), the text of the user’s profile description and posts are first converted to integer labels, then dense $E \times L$ matrices, where E is the embedding dimension, and L is the maximum sequence length.
- The collection of post and description embedding matrices are then given as input to the **Convolutional Neural Network** that creates a *summary embedding* for each piece of text.
- The post *summary embeddings* are then passed as input alongside the time when they were posted to the **Recurrent Neural Network**. This model then produces a *post corpus embedding* that summarizes the information of all posts created by the user into a single vector representation.
- Finally, the *post corpus embedding*, the profile description *summary embedding* and the user’s z-score are given

to the **Multilayer Perceptron** that returns a single value between 0 and 1, indicating the probability that the user is a bot.

Word Embeddings The tokenization and embedding is the first step of the pipeline, and arguably the most important; word embeddings capture the complex meanings of words within a wide variety of contexts and define the inputs around which the other models train their parameters. I elected to use the embeddings and tokenizer taken from DeepSeek-R1-Distill-Qwen-1.5B model (DeepSeek-AI 2025) because it is a model with proven performance on various natural language benchmarks that is additionally lightweight enough for the embeddings to be usable in the resource-limited test environment. However, these advantages do not mean that there are no shortcomings to using pre-trained embeddings from an LLM. In the case of the competition, the fact that inputs are social media posts and profile descriptions means that there are a multitude of strange tokens spread throughout the datasets, such as emojis, links or mentions. Unfortunately, all of these tokens are mapped to the **Unknown Token** value by the tokenizer, which means that the embeddings fail to capture the information contained in these elements, even though they are often critical to the message they are contained in. To circumvent this problem, I applied a pre-processing step that converted all emojis to a plain-text description of what they represent, and links to the string *"link"*. This process allowed all of the elements of the input to be represented with little loss of information and without the need to add new tokens to the tokenizer, a process that would have required the embedding layer to be re-trained.

Convolutional Neural Network A *Convolutional Neural Network (CNN)* is rarely used in text classification due to the limitations of its architecture. Indeed, by design, CNNs operate by taking weighted averages over neighboring values, then pooling to condense information and reduce noise, this process allows for very strong summarizations over small distances, but fails to represent long-distance relationships. This weakness is much less present in the case of a short-form dataset such as posts from Twitter. Given the fact that the majority of the posts in the dataset contained 20 tokens or less, and the largest filter size used in my CNN implementation was 5, the effect of losing long-distance relationships between tokens does not significantly impact the downstream accuracy of the model on the classification task.

Recurrent Neural Network The principal weakness of a *Recurrent Neural Network (RNN)* is the limit of its ability to store information of past states. This weakness is largely avoided in my implementation due to the fact that the length of the input sequences is the number of posts created by a given user. In the competition datasets, the average number of posts per user is less than 30. This means that there is less opportunity for information to be lost compared to the most common use case, which is to use an RNN as a sentence-level summarization model, where it may struggle to accurately represent longer sentences.

Multilayer Perceptron Finally, the *Multilayer Perceptron (MLP)* has a simple goal when compared to the other models: determine which combinations of input values map to bots, and which ones map to human users. This final model acts as the final summarization of all the outputs of previous models concatenated to the input values that do not require any additional processing, such as the user's z-score.

Prior Approaches In earlier iterations of the model, there are 3 main decisions that I made that, when improved upon, yielded large increases in model performance:

- word2vec embeddings: The first iterations of my detector made use of word embeddings that were not pre-trained. Instead, I trained my own word embeddings from the text in the provided dataset. The problems with this approach were the lack of depth due to the small embedding dimension, and the sensitivity to *out of context (OOC)* tokens.
- Feeding all inputs into the RNN: When I first incorporated the RNN into my model's architecture, I would repeat the *summary embedding* of the profile description alongside the post time and the z-score of the user for each post. This redundancy created unnecessary noise that limited the accuracy of the overall detector.
- Using the whole dataset during model training: Earlier versions of my detector were trained on the entire train split of the dataset. However, due to the overwhelming majority of users being human, the model learned to simply classify every user as a human, as misclassifying every single bot yielded a smaller error than attempting to properly classify the data and making some mistakes. To avoid this issue, I began randomly eliminating a large proportion of the human user posts in my dataset prior to training to enforce that a statistically relevant proportion of the training data would be bot users.

3.3 Bot User Architecture

Architecture Overview In the context of the competition, the goal of every bot was simply to remain undetected at all costs. There were certain requirements put in place: a minimum number of posts and users, but there were otherwise no other restrictions. For my *Social Media Bots*, I elected to use a combination of LLM and naive text generation methods alongside functions that incorporated randomness into the generation schedule to emulate human interaction patterns.

As illustrated in Algorithm 1, each bot user generates their content within a certain scheduled period of time. Though it is not shown, when a post is added to the output list, a random post time is assigned to it by taking the post time of the original post, then adding a random wait time to simulate a human user reacting to a real post while scrolling through their feed.

The tasks of the bot user profile also include generating a profile description. For this, I also used an LLM at test time to generate a profile for every bot user.

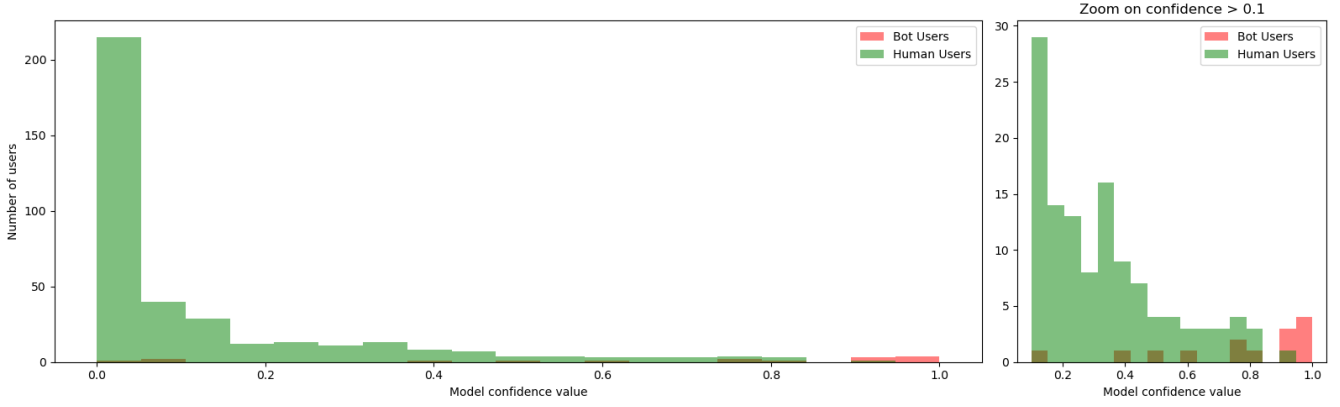


Figure 2: Histogram of human vs. bot users based on model confidence

Algorithm 1: Bot User Post Generation

Input: List of human posts prior to content injection

Parameter: Bot user .json configuration file

Output: List of posts to be injected into the content stream

```

1: Let startTime = config.startTime + random noise.
2: Let scrollDuration = config.expectedDuration + random noise.
3: Let output = empty list.
4: while unseen posts or currentTime < startTime + scrollDuration do
5:   Let currentPost = next post.
6:   if detect interest in currentPost then
7:     Generate new post.
8:     Append new post to output.
9:   else
10:    if random condition then
11:      Append currentPost to output.
12:    end if
13:  end if
14: end while
15: return solution

```

LLM Prompting To receive higher-quality generated posts and profile descriptions from the LLM, I made use of several prompting techniques such as prompt templating and few-shot learning. Prompt templating increases consistency between runs by creating a static formatting for prompts sent to the model. Few-shot learning, on the other hand, shows the model a handful of example outputs inside the prompt, this technique is shown to increase the quality of outputs with little extra effort (Dong et al. 2024).

Copying Other Users One of the best approaches to avoiding detection that I discovered was simply to instruct the bot user to occasionally take a random post made by a human user, regardless of topic or style, and include it in the output posts generated by the LLM. When set to the correct frequency, this technique adds enough content to the profile that models trained to detect LLM-generated content will

Bot	Detected	Not Detected	Average Confidence
Bot 1	0	4	24
Bot 2	1	3	27
Bot 3	2	2	45

Table 1: Performance of detection evasion by bot users.

flag as human-generated that the bot will go undetected. The main disadvantage of this technique is that any detector that is explicitly searching for repeated content will easily find the bot. However, comparing all posts amongst themselves is a very computationally costly operation, and thus was not used by any of the competitors in their detectors.

4 Results and Discussion

4.1 Detector Performance

For the final test session of the competition, I elected to use the detector model with the architecture described previously. I selected a CNN hidden dimension of 3 times the embedding dimension, an RNN hidden dimension of 2 times the embedding dimension and a 3-layer MLP with an input layer of size $5 \times E$, a hidden dimension of $2 \times E$ and an output dimension of 1. I also elected to use a detection threshold of 0.81 as, during testing before the session, I had determined that it was the ideal threshold to keep the false positive rate below 1% whilst also having a high detection rate. Indeed, in Figure 2 we can see the distribution of predicted confidence scores for human and bot users on the test data, where there is a clear gap at around 0.85 where predictions change from being a mix of bots and humans to mostly bots. Observing the data more closely, the 99th percentile prediction on a human user was 0.775. However, as for elements to improve upon, the true positive rate sat at exactly 50%, meaning that, of the 16 total bot users, My detector only flagged 8. Had I decided to use a threshold of 0.5, the number of detections would have risen to 12, however the false positive rate would have followed, climbing to over 6%, which is not an acceptable outcome. Overall, I consider the result to be satisfactory given my goal of minimizing the false positive rate.

4.2 Bot User Performance

I registered 3 different bot users to the competition, all of them using the architecture provided above. Table 1 details how many detectors (including my own) they managed to evade, alongside the average of non-zero confidence scores attributed to each bot. These results are a clear demonstration of how difficult it can be to detect a *Social Media Bot* in the post-LLM world. In the case of the best-performing bot, its configuration .json file remained unchanged from the first week of the competition to the end, and yet it was not flagged by any detector, including my own, which only attributed to it a confidence of 51.

5 Conclusion

The goal of this competition was to gain new insights into how *Social Media Bots* behave and how better to detect them. This paper presents a new hybrid model for detecting *Social Media Bots* on Twitter based as well as an exploration of potential methods that could be exploited by malicious actors to make their bots harder to detect. The results of the model in the context of the competition are indicative that an acceptable detection rate can be achieved while simultaneously keeping a low false positive rate on a dataset representative of a real-world social media platform. However, certain bots remained very difficult to detect despite being aware of all of the techniques used to produce their output, a result that indicates there is still much room for progress in the field of bot detection.

A Acknowledgments

I am very grateful for the opportunity offered to me by Prof. Derek Ruths to participate in this project and for the feedback and guidance provided over the course of the project. I would also like to extend my thanks to the amazing support team that worked tirelessly on the competition environment.

References

- DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.
- Dong, Q.; Li, L.; Dai, D.; Zheng, C.; Ma, J.; Li, R.; Xia, H.; Xu, J.; Wu, Z.; Liu, T.; Chang, B.; Sun, X.; Li, L.; and Sui, Z. 2024. A Survey on In-context Learning.
- Ng, L.; and Carley, K. 2025. A global comparison of social media bot and human characteristics. *Nature*.
- NSICOP. 2024. Special Report on Foreign Interference in Canada’s Democratic Processes and Institutions.